JUG - AUBAY

AI in Practice

Orchestrating Sagas with LangChain4j

Pedro Santos

LinkedIn

Agenda

PROJECT 1

LangChain4j (Stock Service) Concepts & Tools

- Intro — Core concepts
- Demo 1 — First Chat Model

github.com/pedrop3/stock

stock

PROJECT 2

Intelligent Sagas with LangChain4j

- Distributed Saga architecture
- Step 1 — MCP in microservices
- Step 2 — OperationsAgent
- Step 3 — SagaComposerAgent
- Step 4 — DataAnalystAgent
- 3 Agents — Lessons learned

github.com/pedrop3/saga-orchestration

saga-orchestration

PROJECT 1 — FOUNDATIONS

LangChain4j

Core Concepts

Models | Agents | Tools | RAG

LangChain4j

The Java SDK for building AI-powered applications

LangChain4j is an open-source Java library that lets you connect LLMs (Large Language Models) to your application — with tools, memory, RAG, agents, and MCP — without writing boilerplate HTTP calls or JSON parsing.

AI Services

Define agents as Java interfaces. @SystemMessage + @UserMessage = working agent. No implementation needed.

Integrations

Gemini, OpenAI, Ollama, Anthropic, Claude. Swap models with one line of code.

Tools & MCP

Expose Java methods as tools. The model decides when to call them. MCP for cross-service tools.

RAG & Memory

pgvector, Chroma, Pinecone. Add semantic search and conversation history in minutes.

Choosing the right tool for production AI in Java

	Python LangChain / LangGraph	Spring AI spring.io/projects/spring-ai	LangChain4j ✦ Recommended
Ecosystem fit	⚠️ New stack — runs aside your Java app	✅ Native Spring — auto-configured & familiar	✅ **Zero friction.**
MCP support	✅ Full (Python SDK) well established	✅ Full since 1.1 GA — client + server boot starters	✅ Full — client + server out of the box
Agent definition	⚠️ Explicit chain construction required	⚠️ @AiService works but needs extra wiring	✅ Interface + @SystemMessage = done. No boilerplate.
Model support	✅ Widest — 50+ providers	✅ Good — major providers covered	✅ 20+ providers — OpenAI, Gemini, Ollama, Bedrock…
RAG / Embeddings	✅ Mature — many vector stores	✅ Good — pgvector, Redis, Pinecone	✅ pgvector, Chroma — works with Ollama
API stability	⚠️ Fast-moving — breaking changes between versions	⚠️ Upgrades feel like rewrites (0.x→1.x churn)	✅ Stable post-1.0 GA SemVer respected

💡 Spring AI 1.1 now has full MCP support. LangChain4j if you want MCP, AiServices proxy magic, and a stable API today.

MODELS

What is a Model?

The AI engine that understands language and generates responses

A model is a mathematical function trained on billions of text examples. You give it text (a prompt) → it predicts the most likely next tokens → you get a response. It has no memory between calls — context window is everything.

✨ Gemini Flash/Pro

Cloud

Google DeepMind — Cloud API

Fast, multimodal, cheap. Gemini Flash for reasoning + tool use. Pro for complex tasks. No local install — just an API key.

```
GoogleAiGeminiChatModel.builder()
  .apiKey(System.getenv("GEMINI_API_KEY
"))
  .modelName("gemini-2.5-flash")
  .build();
```

🦙 Ollama (local)

Local

Open-source — runs on your machine

Run models locally — no API key, no cost, no data leaves your machine. Ideal for embeddings and privacy-sensitive workloads.

```
OllamaChatModel.builder()
  .baseUrl("http://localhost:11434")
  .modelName("llama3")
  .build();
```

🔗 LangChain4j abstraction

Portable

Swap models with one line

Both Gemini and Ollama implement the same ChatModel interface. Switch providers without touching your agent code.

```
// Same agent — different model
ChatModel model = isProduction
  ? geminiModel
  : ollamaModel;
```

OLLAMA

Ollama — AI models on your laptop

One command to run Llama, Mistral, nomic-embed-text and 100+ models locally

1 Install Ollama

```
brew install ollama
# or: curl -fsSL https://ollama.ai/install.sh | sh
```

2 Pull a model

```
ollama pull llama3
ollama pull nomic-embed-text   # for embeddings
```

3 Start the server

```
ollama serve
# API runs at http://localhost:11434
```

4 Use in LangChain4j

```
OllamaEmbeddingModel.builder()
  .baseUrl("http://localhost:11434")
  .modelName("nomic-embed-text")
  .build();
```

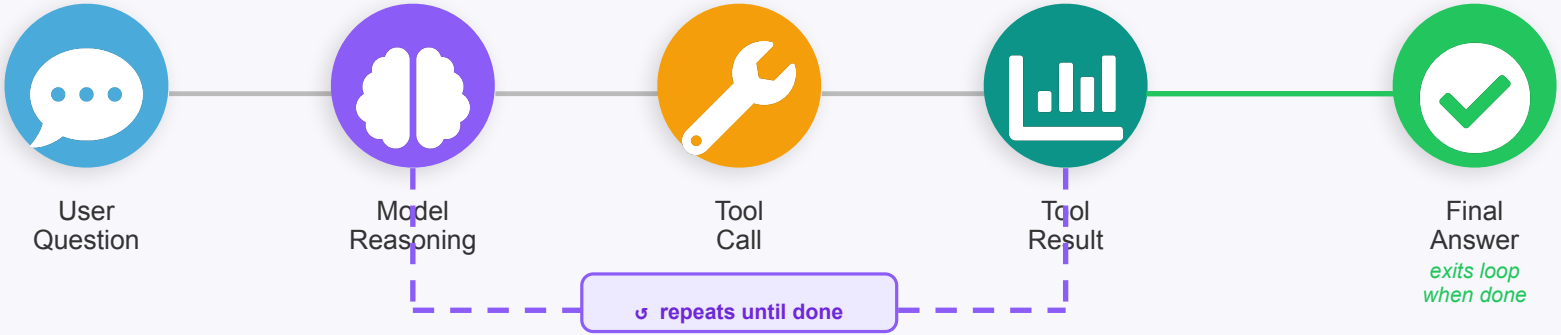
💡 Ollama is perfect for embeddings (nomic-embed-text) — free, fast, private. Use Gemini for complex reasoning tasks.

AI AGENTS

What is an Agent?

A model that can reason, plan, and call tools to complete a task

An agent is a loop: the model receives a task → decides which tools to call → calls them → observes results → decides the next action → repeats until the task is done.




```
public interface DataAnalystAgent {

    @SystemMessage("You are a saga analyst. Use only MCP tools.")
    String analyze(@UserMessage String question);
}
// LangChain4j creates the implementtion at runtime
DataAnalystAgent agent = AiServices.builder(DataAnalystAgent.class)
    .chatModel(gemini).toolProvider(mcpProvider).build();
```

Model generates text → Tool bridges to real data → Agent loops until task is complete

TOOLS / FUNCTION CALLING

What are Tools?

Java methods the model can invoke to get real-world data

Tools bridge the gap between LLM text generation and real data. The model sees your method signature and description — it decides WHEN to call it.

```
// Annotate any method with @Tool
public class OrderTools {

    @Tool("Returns stock for a product. Use to check availability.")
    public int getStock(@P("Product code") String code) {
        return inventoryRepo.findByCode(code).getAvailable();
    }

    @Tool("Returns the fraud risk score for an order.")
    public String getFraudScore(double amount, String type, int hour) {
        return fraudService.calculate(amount, type, hour);
    }
}

// Register — the model automatically decides when to call
AiServices.builder(MyAgent.class)
    .tools(new OrderTools())
    .build();
```

1 You define a @Tool method

Any Java method with a description. The model reads the method name, params, and @Tool description.

2 Model sees the tool signature

LangChain4j sends the method signature to Gemini as a functionDeclaration in the request.

3 Model decides to call it

When the answer requires live data, the model outputs a functionCall — LangChain4j intercepts it.

4 Java executes the method

Your actual business logic runs. The result is sent back to the model as a functionResponse.

🔗 MCP = @Tool but over HTTP/SSE — any agent can connect

TOKENS

What are Tokens — and why do they cost money?

Every word you send and receive is measured in tokens - and every token has a cost or a compute price

What is a Token?

Orches | trating | Sagas | with

Lang | Chain | 4j

≈ 7 tokens

- ~1 token ≈ ¾ of a word in English
- punctuation, spaces = separate tokens
- non-English text uses more tokens
- 1000 tokens ≈ 750 words of prose

How Pricing Works

Input tokens

(your prompt + history)

~$0.075 / 1M

Output tokens

(model response)

~$0.30 / 1M

Thinking tokens

(internal reasoning)

~$3.50 / 1M

⚠ Thinking tokens (Ollama .think(true)) count as INPUT on cloud APIs — can 10x your cost per call

Your Optimisations

.numPredict(512)
Caps output tokens → limits cost per call

.numCtx(32768)
Large context = more input tokens per turn

.temperature(0.0)
Deterministic → no wasted sampling attempts

.repeatPenalty(1.2)
Avoids loops → shorter responses

TokenUsageListener
Tracks input/output tokens per call

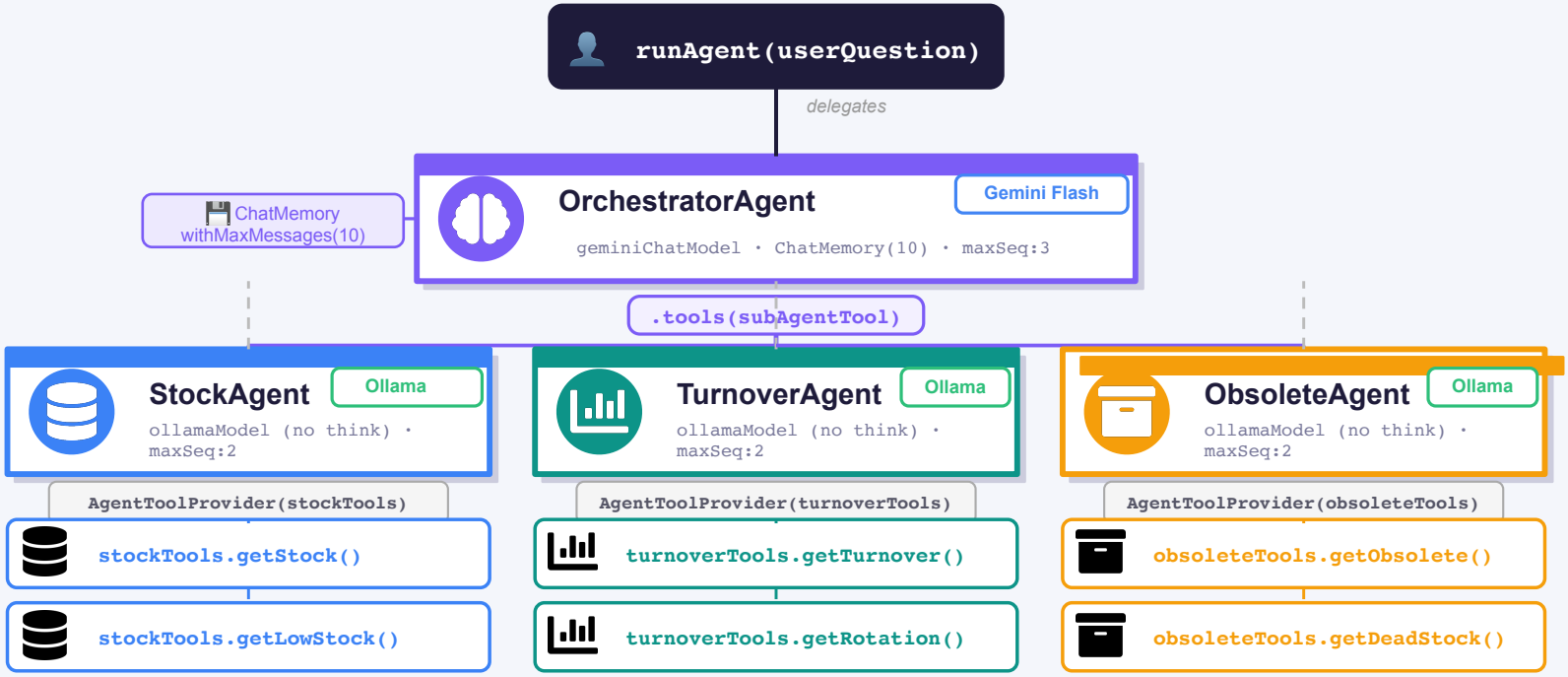
.think(true) [Ollama]
Free locally — costly on cloud APIs

Input + Output + Thinking tokens = Total cost • numPredict · numCtx · repeatPenalty · TokenUsageListener = your control levers

AI AGENTS

Multi-Agent Architecture with LangChain4j

OrchestratorAgent (Gemini) routes tasks via SubAgentTools — 3 Ollama sub-agents, each isolated with AgentToolProvider



Gemini orchestrates → SubAgentTools(@Tool) wraps sub-agents → each Ollama sub-agent uses AgentToolProvider(ownTools only)

PROJECT 2

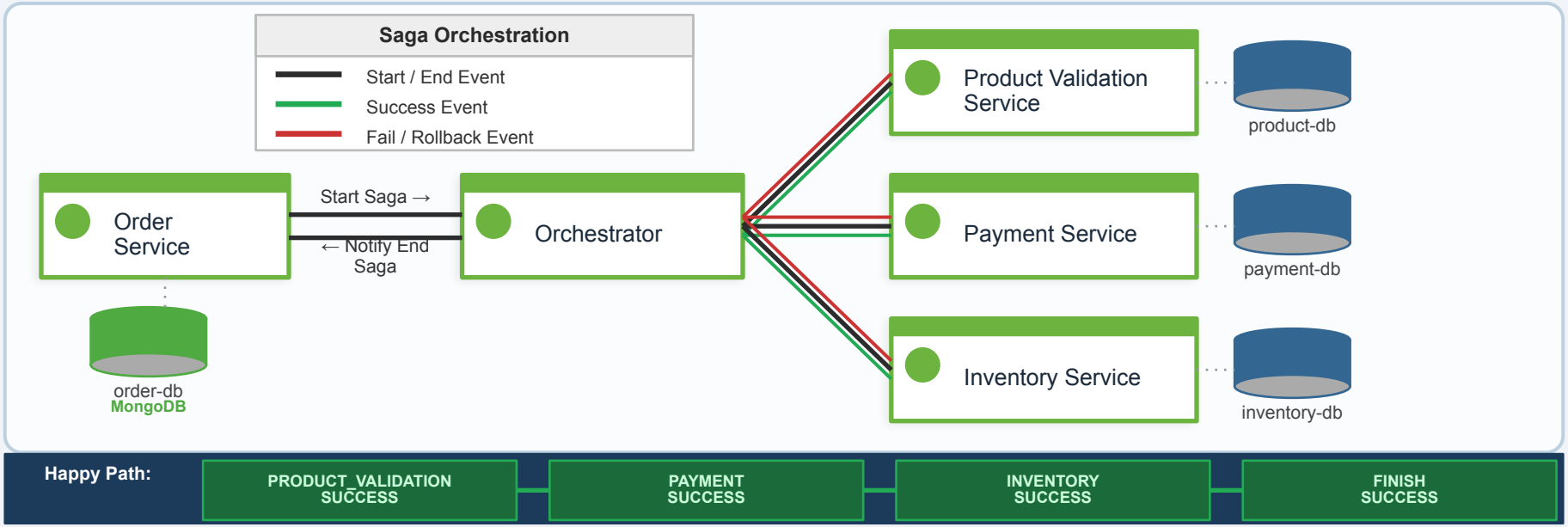
Intelligent Sagas

with LangChain4j

DEMO 3

Saga Orchestration - Order Flow

Spring Boot | Kafka | MongoDB | PostgreSQL



Happy Path: PRODUCT_VALIDATION SUCCESS → PAYMENT SUCCESS → INVENTORY SUCCESS → FINISH SUCCESS

Saga Pattern: Compensating transactions ensure consistency across microservices without distributed transactions.

DEMO 3

Saga Orchestrator - State Transitions

For each service response, the Orchestrator decides the next topic to publish based on status

Service Origin	Status	Topic
ORCHESTRATOR	SUCCESS	PRODUCT_VALIDATION_SUCCESS
ORCHESTRATOR	FAIL	FINISH_FAIL
PRODUCT_VALIDATION_SERVICE	ROLLBACK_PENDING	PRODUCT_VALIDATION_FAIL
PRODUCT_VALIDATION_SERVICE	FAIL	FINISH_FAIL
PRODUCT_VALIDATION_SERVICE	SUCCESS	PAYMENT_SUCCESS
PAYMENT_SERVICE	ROLLBACK_PENDING	PAYMENT_FAIL
PAYMENT_SERVICE	FAIL	PRODUCT_VALIDATION_FAIL
PAYMENT_SERVICE	SUCCESS	INVENTORY_SUCCESS
INVENTORY_SERVICE	ROLLBACK_PENDING	INVENTORY_FAIL
INVENTORY_SERVICE	FAIL	PAYMENT_FAIL
INVENTORY_SERVICE	SUCCESS	FINISH_SUCCESS

SUCCESS → next topic | ROLLBACK_PENDING → compensation topic | FAIL → FINISH_FAIL

DEMO 3

Kafka Topics - Service Mapping

Each service produces and/or consumes specific Kafka topics to coordinate the Saga flow

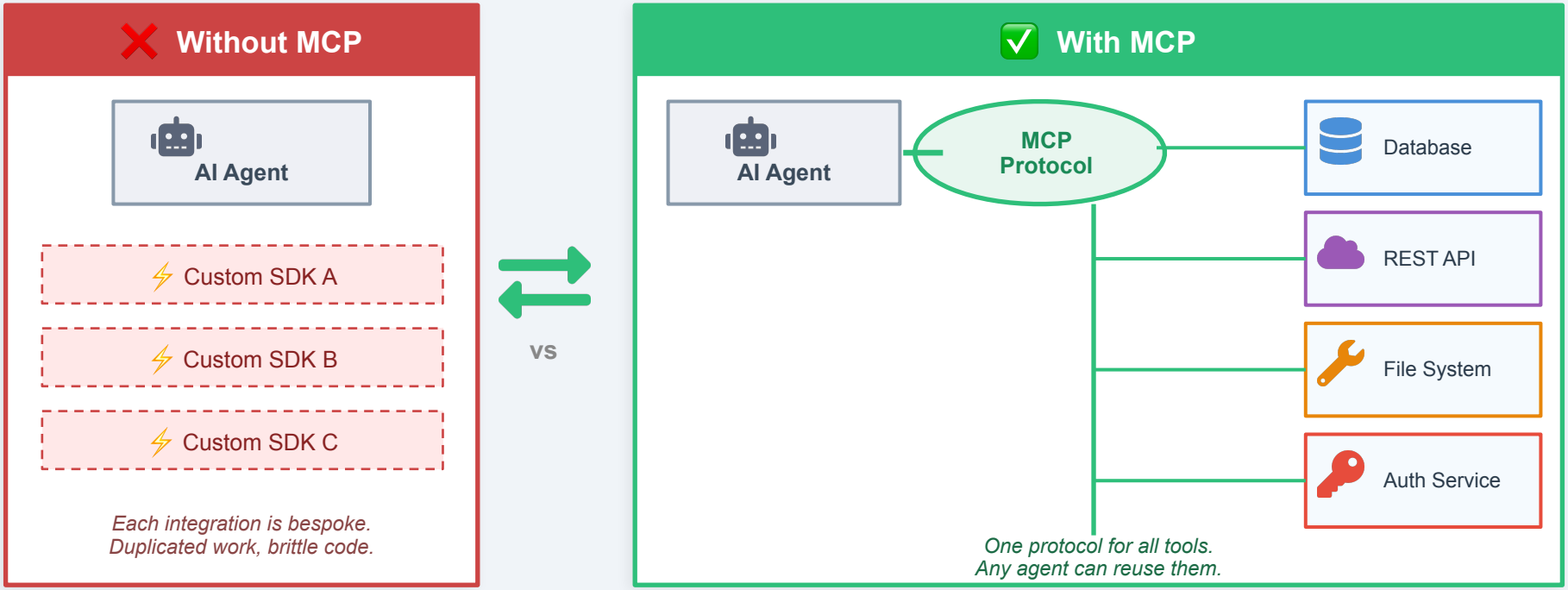
Service	Topic	Type
order-service	notify-ending	consumer
order-service	start-saga	producer
orchestrator	orchestrator	consumer
orchestrator	finish-success	producer/consumer
orchestrator	finish-fail	producer/consumer
orchestrator	notify-ending	producer
product-validation-service	product-validation-success	consumer
product-validation-service	product-validation-fail	consumer
product-validation-service	orchestrator	producer
payment-service	payment-success	consumer
payment-service	payment-fail	consumer
payment-service	orchestrator	producer
inventory-service	inventory-success	consumer
inventory-service	inventory-fail	consumer
inventory-service	orchestrator	producer

start-saga → orchestrator → product-validation | payment | inventory → finish-success / finish-fail → notify-ending

CONCEITO

MCP - Model Context Protocol

A standard protocol that lets AI agents discover and call external tools — like USB for AI



MCP = standard JSON-RPC over HTTP/SSE • Tool discovery • Stateless calls • Any language, any agent

DEMO 3

MCP - Model Context Protocol

@Tool (local)

- Defined in the same JVM
- Tightly coupled to agent
- No network overhead
- Only this agent can use it

MCP Tool (remote)

- Runs in another service
- Any agent can connect
- Exposed via HTTP/SSE
- Standard JSON-RPC protocol

MCP Server — Spring Boot

```
McpServer.sync(transport)
  .serverInfo("order-mcp", "1.0.0")
  .tools(getOrderById(),
listRecentEvents())
  .build();
```

MCP Client — Agent side

```
McpToolProvider provider =
McpToolProvider.builder()
  .mcpClients(List.of(orderMcpClient))
  .build();

AiServices.builder(Agent.class)
  .toolProvider(provider).build();
```

CONCEPT

RAG -Retrieval Augmented Generation

Combining the power of LLMs with external, up-to-date knowledge

Plain LLM

- Knowledge limited to training cutoff date
- No access to private or internal data
- Hallucinates facts when it doesn't know
- Generic context, no domain expertise

How RAG Works

1. User question arrives at the system
2. Retrieve relevant docs from vector DB
3. Context is injected into the LLM prompt
4. LLM generates a grounded answer

RAG Benefits

- ✅ Always up-to-date answers without retraining the model
- 🔒 Access to private and proprietary data
- 📎 Answers backed by citable, verifiable sources
- ⚡ Fewer hallucinations with grounded context

`RAG = Retrieve relevant knowledge → Augment the prompt → Generate a precise answer`

RAG

RAG - Retrieval Augmented Generation

Give the model access to your private data without retraining

❌ The Problem

LLMs are frozen in time - they don't know your saga events, your order history, or your internal docs.

✅ The Solution: RAG

Before asking the model, search your database for relevant context. Inject it into the prompt. The model answers with YOUR data.


```
// 1. Store your data as embeddings
embeddingStore.add(embeddingModel.embed(text), TextSegment.from(text));

// 2. Find the 5 most relevant entries at query time
List<EmbeddingMatch<TextSegment>> matches =
    embeddingStore.findRelevant(embeddingModel.embed(question), 5);

// 3. Inject into the prompt — model answers with YOUR data
String context = matches.stream().map(m -> m.embedded().text())
    .collect(Collectors.joining("\n"));
```

CONTEXT

Distributed Saga Architecture

ai-saga-agent

port: 8099

Gemini · MCP Client · Redis · pgvector

order-service

port: 3000

MongoDB / Kafka /MCP Server

orchestrator

port: 8050

MongoDB / Redis · Kafka

product-validation

port: 8090

PostgreSQL · Kafka · MCP Server

payment-service / fraud-service

port: 8091

PostgreSQL · Kafka · MCP Server

inventory-service

port: 8092

PostgreSQL · Kafka · MCP Server

STEP 1

MCP in Microservices

branch: 1-step-adding-mcp

```
// Each microservice exposes its own MCP server
@Bean
public McpSyncServer mcpServer(
        HttpServletSseServerTransportProvider transport,
        PaymentService paymentService,
        FraudValidationService fraudService) {

    return McpServer.sync(transport)
        .serverInfo("payment-mcp", "1.0.0")
        .capabilities(ServerCapabilities.builder().tools(
true).build())
        .tools(
            getPaymentStatus(paymentService),
            getRefundRate(paymentService),
            getFraudRiskScore(fraudService)   // ← real
logic reused
        )
        .build();
}
```

order-service

getOrderById · listRecentEvents · getLastEventByOrder

payment-service

getPaymentStatus · getRefundRate · getFraudRiskScore

inventory-service

getStockByProduct · getLowStockAlert · checkReservationExists

product-validation

checkProductExists · checkValidationExists · listCatalog

The 3 Agents — How They Work Together

💬 DataAnalystAgent

Trigger: HTTP: POST /api/agent/analyze

Answers operational questions in natural language

→ Human-readable analysis

🔍 OperationsAgent

Trigger: Kafka: notify-ending (FAIL)

Automatically diagnoses saga failures using RAG

→ pgvector + PostgreSQL

🧩 SagaComposerAgent

Trigger: Scheduled (60s / 30min)

Decides optimal step order per customer profile

→ Redis: saga-plan:{profile}

STEP 4

DataAnalystAgent — Natural Language Queries

🌿 branch: 5-step-ai-improve-fluxo

👤 List the 5 most recent failed sagas and assess their fraud risk.

🤖 Order 69b6c29f → Payment rejected (R$932.80 > R$500 limit). Fraud score: 12/100 APPROVED. No compensation — payment was never processed.

👤 Which products are at risk of running out of stock?

```
@SystemMessage("""
    You are a data analyst for distributed sagas. Use only MCP tools.

    Workflow: Extract N → listRecentEvents(N+10) → filter FAIL
    → getOrderById → getFraudRiskScore → report N results only.
    """)
String analyze(@UserMessage String question);
```

STEP 2

OperationsAgent - Auto-Diagnosis

🌿 branch: 3-step-ai-agent-operation

Kafka
notify-ending

Embed
event data

RAG search
similar cases

Gemini Pro
diagnosis

Store in
pgvector + DB

```
@KafkaListener(topics = "${spring.kafka.topic.notify-ending}",
               groupId = "ai-agent-group")
public void onSagaEnded(String payload) {
    Event event = jsonUtil.toEvent(payload).orElseThrow();

    // Vectorize ALL events for learning
    vectorizeEvent(event);

    // Diagnose only failures
    if (event.getStatus() == FAIL) {
        String diagnosis = operationsAgent.diagnose(buildContext(event));
        sagaDiagnosticRepository.save(new SagaDiagnostic(event, diagnosis));
    }
}
```

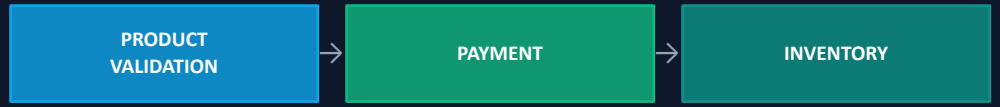
STEP 3

SagaComposerAgent — Dynamic Planning

🌿 branch: 4-step-ai-agent-saga-plan

new:high-value

vip:any




```
// Runs every 60s (dev) / 30min (prod)
@Scheduled(fixedDelayString = "${saga.composer.interval:60000}")
public void recomputePlans() {
    for (String profile : profiles) {
        String json = sagaComposerAgent.compose(buildProfileContext(profile));
        redisTemplate.opsForValue().set("saga-plan:" + profile, json, 35, MINUTES);
    }
}
```

LESSONS

Lessons Learned

✅

MCP > @Tool for microservices

Reuse business logic across any agent without coupling

⚠️

SystemMessage alignment is critical

Tools described in prompt that don't exist → agent fails silently

📊

JSON responses win over key=value

ObjectMapper.writeValueAsString() — one line, zero bugs

🎯

Workflow instructions > tool descriptions

Tell the agent HOW to use tools, not just WHAT they do

🔢

maxOutputTokens matters

1024 is not enough for 5 sagas + fraud scores — use 4096

🚀

Virtual threads are essential

spring.threads.virtual.enabled=true — parallel MCP calls at no cost

Q&A

Questions? Let's talk.

🔗 github.com/pedrop3/saga-orchestration

🔗 github.com/pedrop3/stock

📦 langchain4j.dev

LinkedIn

Training Evaluation